

AFRILANG Stdlib

std/c/netsubnet

Français. Bibliothèque AFRILANG 'std/c/netsubnet' (niveau complex, catégorie complex). Elle expose 7 fonction(s) : subnLen, subnUpper, subnLower, subnTrim, subnSplit, subnJoin, subnReplace.

```
'import "std/c/netsubnet"' · 'use netsubnet'
```

```
- 'subnLen(s text) → number' - 'subnUpper(s text) → text' - 'subnLower(s text) → text' - 'subnTrim(s text) → text' - 'subnSplit(s text, delim text) → list text' - 'subnJoin(parts list text, delim text) → text' - 'subnReplace(s text, from text, to text) → text'
```

English. AFRILANG library 'std/c/netsubnet' (tier complex, category complex). Exports 7 function(s): subnLen, subnUpper, subnLower, subnTrim, subnSplit, subnJoin, subnReplace.

```
'import "std/c/netsubnet"' · 'use netsubnet'
```

```
- 'subnLen(s text) → number' - 'subnUpper(s text) → text' - 'subnLower(s text) → text' - 'subnTrim(s text) → text' - 'subnSplit(s text, delim text) → list text' - 'subnJoin(parts list text, delim text) → text' - 'subnReplace(s text, from text, to text) → text'
```

Catégorie / Category	Modules complex (c/)
Niveau / Tier	complex
Fonctions / Functions	7

June 16, 2026

Contents

Français

1. Vue d'ensemble

Bibliothèque AFRILANG 'std/c/netsubnet' (niveau complex, catégorie complex). Elle expose 7 fonction(s) : subnLen, subnUpper, subnLower, subnTrim, subnSplit, subnJoin, subnReplace.

```
'import "std/c/netsubnet"' · 'use netsubnet'
```

```
- `subnLen(s text) → number`
- `subnUpper(s text) → text`
- `subnLower(s text) → text`
- `subnTrim(s text) → text`
- `subnSplit(s text, delim text) → list text`
- `subnJoin(parts list text, delim text) → text`
- `subnReplace(s text, from text, to text) → text`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/netsubnet"
use netsubnet
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés – graphes, NLP, réseaux complexes.

3. Référence API

- subnLen(s text) → number•
- subnUpper(s text) → text•
- subnLower(s text) → text•
- subnTrim(s text) → text•
- subnSplit(s text, delim text) → list•
- subnJoin(parts list text, delim text) → text•
- subnReplace(s text, from text, to text) → text

4. Exemples d'utilisation

```
import "std/c/netsubnet"
use netsubnet
```

```
create result = subnLen(/* args */)
say result
```

```
create result = subnUpper(/* args */)
say result
```

```
create result = subnLower(/* args */)
say result
```

```
create result = subnTrim(/* args */)
say result
```

```
create result = subnSplit(/* args */)
say result
```

```
create result = subnJoin(/* args */)
say result
```

5. Bonnes pratiques

- Préférez ``import "std/c/netsubnet"`` en tête de fichier.
- Lancez ``afriLang check`` avant de livrer.
- Combinez avec les modules stdlib de la même catégorie.
- Voir `/stdlib/` pour le source live et les démos playground.
- Signalez les problèmes sur GitHub si une export manque.

6. Annexe — source

module netsubnet

```
function subnLen(s text) returns number
end
```

```
function subnUpper(s text) returns text
end
```

```
function subnLower(s text) returns text
end
```

```
function subnTrim(s text) returns text
end
```

```
function subnSplit(s text, delim text) returns list text
end
```

```
function subnJoin(parts list text, delim text) returns text
end
```

```
function subnReplace(s text, from text, to text) returns text
end
end
```

English

1. Overview

AFRILANG library 'std/c/netsubnet' (tier complex, category complex). Exports 7 function(s): subnLen, subnUpper, subnLower, subnTrim, subnSplit, subnJoin, subnReplace.

```
'import "std/c/netsubnet"' · 'use netsubnet'
```

```
- `subnLen(s text) → number`
- `subnUpper(s text) → text`
- `subnLower(s text) → text`
- `subnTrim(s text) → text`
- `subnSplit(s text, delim text) → list text`
- `subnJoin(parts list text, delim text) → text`
- `subnReplace(s text, from text, to text) → text`
```

2. Installation & import

Add the module to your program:

```
import "std/c/netsubnet"
use netsubnet
```

Tier: complex · Category: Complex modules (c/)
Advanced domains – graphs, NLP, complex networks.

3. API reference

- subnLen(s text) → number•
- subnUpper(s text) → text•
- subnLower(s text) → text•
- subnTrim(s text) → text•
- subnSplit(s text, delim text) → list•
- subnJoin(parts list text, delim text) → text•
- subnReplace(s text, from text, to text) → text

4. Usage examples

```
import "std/c/netsubnet"
use netsubnet
```

```
create result = subnLen(/* args */)
say result
```

```
create result = subnUpper(/* args */)
say result
```

```
create result = subnLower(/* args */)
say result
```

```
create result = subnTrim(/* args */)
say result
```

```
create result = subnSplit(/* args */)
say result
```

```
create result = subnJoin(/* args */)
say result
```

5. Best practices

- Prefer ``import "std/c/netsubnet"`` at the top of your file.
- Run ``afrilang check`` before shipping.
- Combine with related stdlib modules from the same category.
- See `/stdlib/` for the live source and playground demos.
- Report issues on GitHub if an export is missing or undocumented.

6. Appendix — source

module netsubnet

```
function subnLen(s text) returns number
end
```

```
function subnUpper(s text) returns text
end
```

```
function subnLower(s text) returns text
end
```

```
function subnTrim(s text) returns text
end
```

```
function subnSplit(s text, delim text) returns list text
end
```

```
function subnJoin(parts list text, delim text) returns text
end
```

```
function subnReplace(s text, from text, to text) returns text
end
end
```

Référence rapide / Quick reference

- Import : `import "std/c/netsubnet"`
- Vérification : `afrilang check`
- Documentation : <https://afrilang.dev/stdlib/std/c/netsubnet/>

Source (excerpt)

```
module netsubnet

function subnLen(s text) returns number
end

function subnUpper(s text) returns text
end

function subnLower(s text) returns text
end

function subnTrim(s text) returns text
end

function subnSplit(s text, delim text) returns list text
end

function subnJoin(parts list text, delim text) returns text
end

function subnReplace(s text, from text, to text) returns text
end
end
```